

LECTURE 13

ER Modeling: Issues and Problems

Fan traps, chasm traps, and the judgment calls real schemas require

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

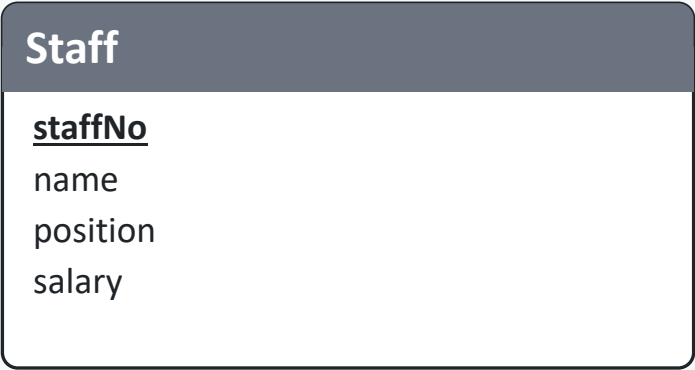
In This Lecture

- 1 Choosing entity types vs. attributes — the single most common beginner confusion
- 2 Choosing relationship types, and spotting one that's hiding a missing entity
- 3 Choosing attributes: simple, composite, single-valued, multi-valued, derived
- 4 Naming conventions — and the concrete query problems synonyms/homonyms cause
- 5 Complex relationships: ternary relationships, relationships with their own attributes
- 6 Structural constraints revisited — the full (min, max) multiplicity notation
- 7 Avoiding redundancy — spotting it, then removing it correctly
- 8 Fan traps and chasm traps — the broken query, and the fix, for each

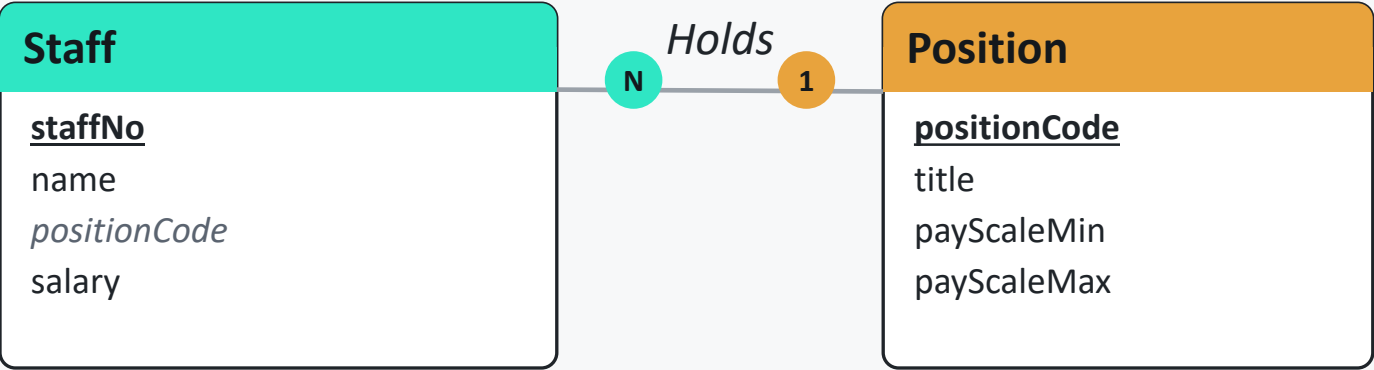
Choosing Entity Types vs. Attributes

- Should “position” be an attribute of Staff, or its own entity type?

Option A — plain attribute



Option B — Position promoted to an entity



THE RULE OF THUMB

Model it as an entity if it needs its own attributes or relationships, or is shared by many rows. Keep it an attribute if it is one atomic fact about one entity. Re-check when requirements grow.

Choosing Relationship Types

- A relationship type is usually named from a verb phrase connecting two entity types (Staff Manages PropertyForRent). Three recurring issues show up when choosing them:

Redundant naming direction

Name the relationship once, in one clear direction. Adding “IsManagedBy” as a SECOND, separate relationship type just models the same fact twice.

A relationship hiding an entity

If a “relationship” needs its own attributes describing neither participant individually, it usually deserves its own identity — see Complex Relationships, this lecture.

The wrong degree

Binary (most common), unary/recursive (Staff Supervises Staff), or ternary+. Force-fitting a 3-way fact into two binary relationships can silently lose information.

PREVIEW — THE FAN TRAP

Collapsing a three-way fact into binary relationships can lose the combination. The Fan Trap section shows how.

Choosing Attributes

Kind	Meaning	Example (Client)
Simple	Cannot be broken down further	telNo
Composite	Made of sub-parts sometimes needed separately	name → first/last
Single-valued	Exactly one value per occurrence	dateRegistered
Multi-valued	Can hold several values at once	preferredContactTimes {}
Derived	Computed, not stored itself	age (from dateOfBirth)

Client — mixed attribute kinds

clientNo
firstName
lastName
telNo
preferredContactTimes {}
dateOfBirth

DERIVED ATTRIBUTES USUALLY AREN'T STORED

age goes stale the moment time passes without an update. Most ER diagrams omit derived attributes entirely (computing age from dateOfBirth in a query) or mark them distinctly, precisely so nobody accidentally maps them to a stored column during Lecture 16's mapping step.

Identifying Keys

- Prefer the candidate key that never changes over the entity's lifetime — cnic is stable; a badge number reissued on transfer is not.
- Prefer the shorter, simpler candidate key when several qualify — a single-attribute key over a composite one, all else equal.
- Prefer a key with no NULL-able components, ever — entity integrity (Lecture 6) forbids NULL primary key components.
- The remaining, unchosen candidate keys don't disappear — they stay in the schema as alternate keys, enforced with UNIQUE.

Staff — two candidate keys

staffNo

cnic

name

salary

staffNo CHOSEN AS PK

Both staffNo and cnic are independently unique. staffNo is chosen as the primary key (short, stable, no NULLs); cnic remains an alternate key enforced by UNIQUE.

Naming Conventions

- A schema with a dozen entity types and fifty attributes is only as usable as its names. Two failure modes recur constantly in real projects:

Synonyms

The same real-world concept named differently in different places — `clientNo` vs. `custId`, both meaning “client identifier.” Makes joins and onboarding harder than they need to be.

Homonyms

The SAME name reused for two genuinely different concepts — “type” meaning House/Flat on `PropertyForRent`, but Full-time/Part-time on `Staff`. Reading a query in isolation gives no clue which.

A PRACTICAL NAMING STANDARD, APPLIED CONSISTENTLY ACROSS THIS COURSE

Entity type names: singular, PascalCase (`Staff`, not `staffs`). Attribute names: camelCase, prefixed with a short recognizable stem where it disambiguates (`staffNo`, `branchNo`, `propertyNo`) — every foreign key carries the SAME name as the primary key it references, which is precisely what makes a foreign key recognizable on sight.

Synonyms: a Concrete Before/After

Before — two names for the same fact

Client	SupportTicket
<u>clientNo</u> firstName	<u>ticketNo</u> custId subject

custId on SupportTicket and clientNo on Client are the SAME identifier under two names.

Why it breaks a real query

A developer writing “list every client and their open tickets” has to already know custId secretly means clientNo — nothing in the schema says so. A naive JOIN on clientNo = custId still works today, but every new hire re-derives this fact from scratch, and a renaming tool that renames clientNo won't touch custId.

After — one name, everywhere

Client	SupportTicket
<u>clientNo</u> firstName	<u>ticketNo</u> <i>clientNo</i> subject

THE FIX

Renaming custId to clientNo costs one migration, once. Leaving it costs every future query author a small re-derivation — that asymmetry is exactly why the naming standard (previous slide) insists a foreign key keeps its referenced primary key's exact name.

Homonyms: a Concrete Before/After

Before — one name, two meanings

PropertyForRent	Staff
<u>propertyNo</u> type street	<u>staffNo</u> type name

type means “House or Flat” on PropertyForRent, but “Full-time or Part-time” on Staff — the same column name, two unrelated domains.

Why it breaks a real query

SELECT type FROM PropertyForRent UNION SELECT type FROM Staff is syntactically valid SQL that silently mixes “House” with “Full-time” in one result column — nothing in the schema stops it, and a report built on it is simply wrong.

After — disambiguated names

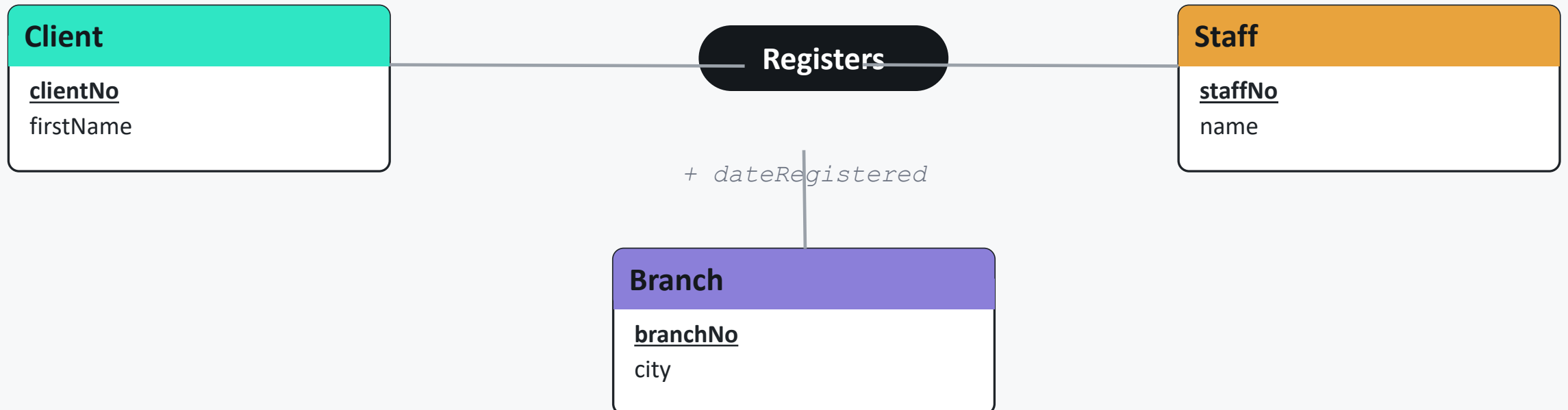
PropertyForRent	Staff
<u>propertyNo</u> propertyType street	<u>staffNo</u> employmentType name

THE FIX

propertyType and employmentType each carry the stem that disambiguates them — exactly the naming standard from the previous slide. A UNION across them now fails obviously (different column names), which is the correct failure mode: loud, not silent.

Ternary Relationships

- A ternary relationship connects three entity types at once — NOT generally the same as three separate binary relationships between the same pairs. Registers: a Client registers interest with a specific Staff member, at a specific Branch, on a given date.

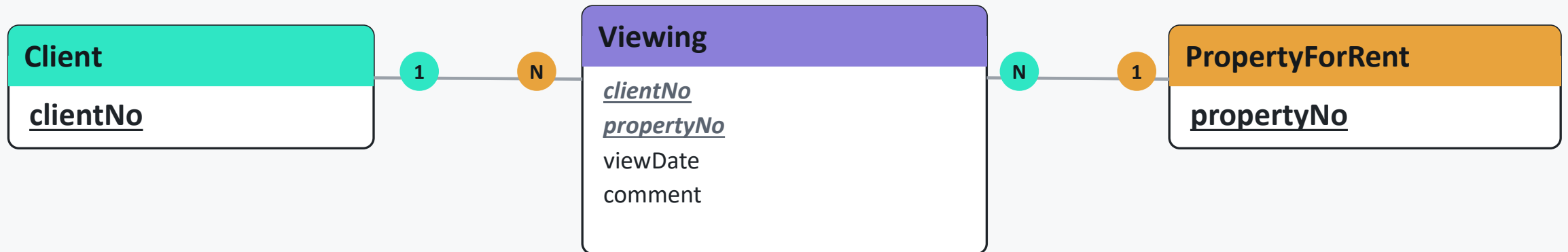


WHY NOT TWO BINARY RELATIONSHIPS

Two binary relationships cannot prove both facts belong to one SAME registration event.

Relationships with Attributes

- Viewing, already used since Lecture 6, is the clearest example in this course's schema: a Client views a PropertyForRent, and the relationship itself carries viewDate and comment — facts about that specific viewing event, not about either participant alone.

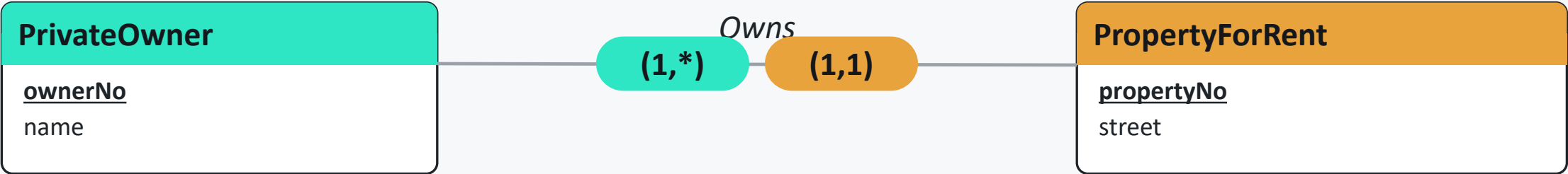


WHEN A RELATIONSHIP NEEDS ITS OWN ATTRIBUTES

Viewing(clientNo, propertyNo, viewDate, comment) — the underlined pair is the composite primary key, each component also a foreign key back into Client and PropertyForRent. Mapped to its own relation (Lecture 16).

Structural Constraints Revisited

Multiplicity	Meaning	DreamHome example
(0,1)	Optional, at most one	Property may have staffNo = NULL
(1,1)	Mandatory, exactly one	Each property has exactly one Branch
(0,*)	Optional, many	Staff may manage zero properties
(1,*)	Mandatory, at least one	A PrivateOwner owns at least one property

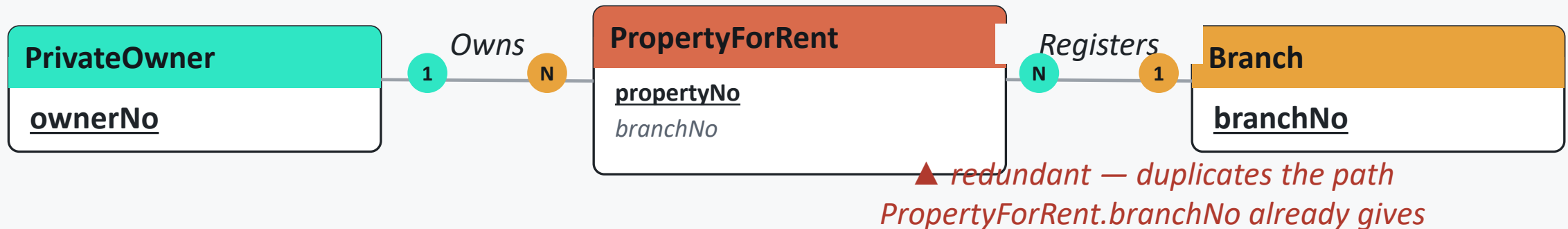


A REAL BUG, NOT A DETAIL

Writing (0,1) on the owner side would wrongly claim a property can legally have no owner at all.

Spotting a Redundant Relationship

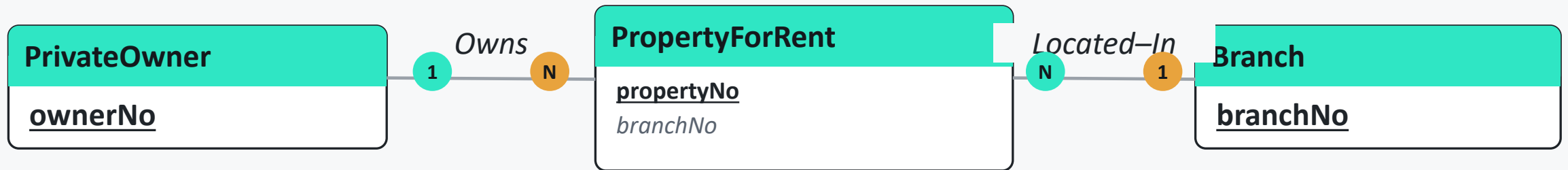
- A relationship is redundant if it conveys a fact already fully derivable by following a different path in the model.
Test: delete this relationship — can I still answer the same question by tracing through others?



THE LOOK-ACROSS TEST

If **PropertyForRent** already carries **branchNo** directly, “which branch handles property P's paperwork” is already answerable. A third relationship claiming owners register with branches adds nothing but a second, possibly inconsistent, place for the same fact to live. Relationships that merely SHARE an entity type (**Manages** and **Owns** both touch **PropertyForRent**) are NOT automatically redundant — only when the SAME fact is representable two different ways.

Redundancy — Corrected



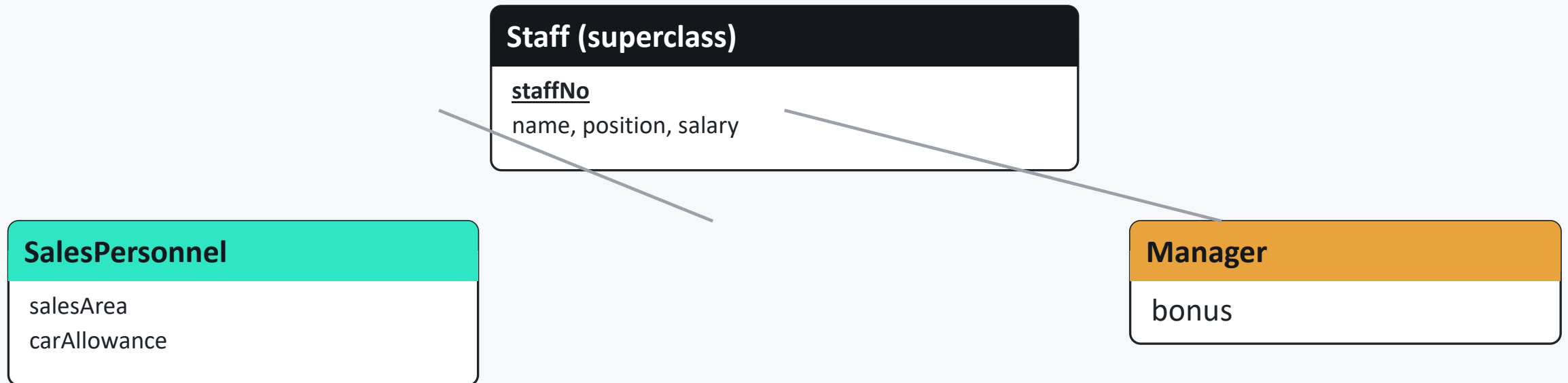
The Registers relationship is simply gone — not replaced, not merged, deleted.

WHAT CHANGED AND WHY

What changed, and why: PropertyForRent.branchNo (from Owns → the property's own Located-In relationship to Branch) already answers “which branch handles this property's paperwork” for every property, every time. Removing Registers doesn't lose any fact the model could answer before — it only removes a SECOND place that same fact could (and eventually would) drift out of sync. This is the general fix for every redundancy case: delete the duplicate path, never try to keep two copies synchronized by hand or by trigger.

A First Taste of Generalization / Specialization

- Staff at a growing agency needs to distinguish SalesPersonnel (salesArea, carAllowance) from Manager (bonus) — both still Staff, sharing staffNo/name/position/salary, but each needs attributes the OTHER doesn't.



PLAIN ER HAS NO CLEAN WAY TO EXPRESS THIS

Plain ER forces either NULLs on most rows or duplicated attributes across unrelated entities. Lecture 14's EER model exists to close this gap.

Fan Trap — the Setup

- A fan trap occurs when two or more 1:N relationships radiate from a SINGLE entity type, and a question needs to relate occurrences on the “many” side of one to the “many” side of the other.



WHY TWO CORRECT 1:N RELATIONSHIPS STILL FAIL

“Which properties does staff member SG14 manage?” cannot be answered — knowing Branch B003 has staff {SG14, SG37} and properties {PA14, PG4, PG36} doesn't say which staff manages which property. Both relationships are individually correct; the diagram simply never recorded the pairing the question needs.

Fan Trap — Walking Through the Broken Query

- Real rows that Branch B003 could store:

Staff

staffNo	branchNo
SG14	B003
SG37	B003

PropertyForRent

propertyNo	branchNo
PA14	B003
PG4	B003
PG36	B003

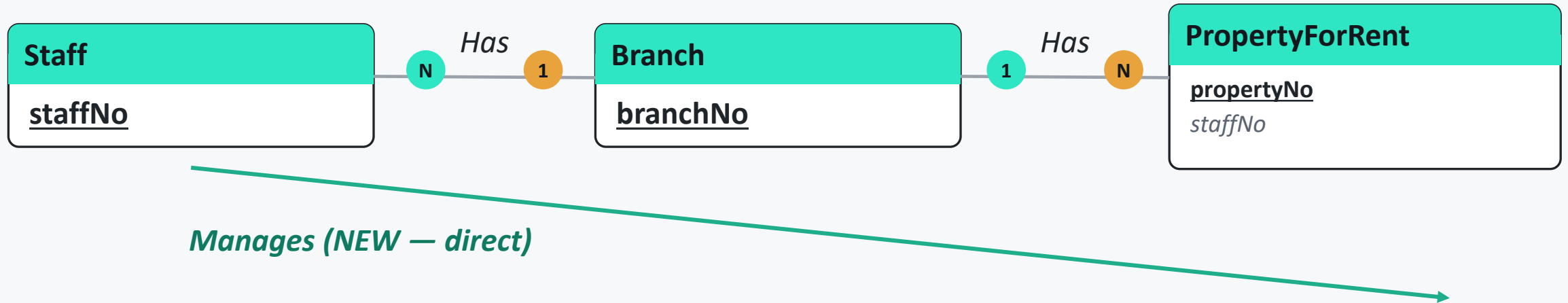
THE WALK-THROUGH

Try “which properties does SG14 manage?” SG14 → B003, and B003 → {PA14, PG4, PG36}. No column pairs a staffNo with a propertyNo, so the join returns all three for BOTH SG14 and SG37.

THE TELL

The symptom: the right row count but the wrong pairing — a report silently says SG37 manages PA14. That is a fan trap: two 1:N relationships fan out from one shared entity.

Fan Trap — Corrected

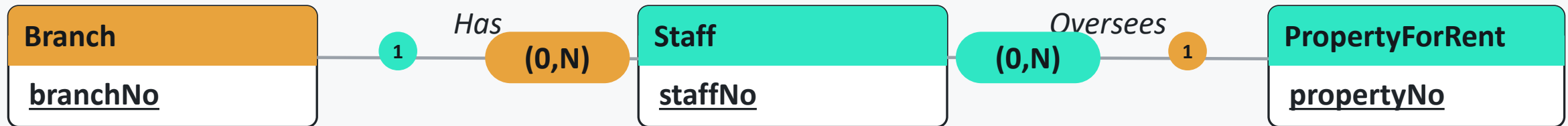


WHAT CHANGED AND WHY

What changed, and why: a NEW direct relationship, Staff Manages PropertyForRent, is added, and PropertyForRent now carries *staffNo* as its own foreign key. The question “which properties does SG14 manage?” is now a single filter — `SELECT propertyNo FROM PropertyForRent WHERE staffNo = 'SG14'` — no join through Branch needed, so there's no longer a shared-entity fan to get lost in. This is exactly the schema Lecture 6 already uses.

Chasm Trap — the Setup

- A chasm trap occurs when a model implies a relationship along a path that includes an OPTIONAL (partial-participation) relationship — so the path exists for some occurrences but is genuinely missing for others.



WHY AN OPTIONAL CHAIN LEAVES A HOLE

“Which properties belong to Branch B009?” cannot always be answered via Branch → Staff → PropertyForRent: if B009's staff all currently oversee zero properties, or B009 itself has zero staff, the path disappears entirely for that branch — even though B009 plausibly still has properties waiting to be assigned. The (0,*) optional minimum on BOTH legs is exactly what creates the gap.

Chasm Trap — Walking Through the Broken Query

- Branch B009 is brand-new — just opened, no staff hired yet, but three properties already assigned to it directly:

Branch		Staff (oversees, via Branch)		PropertyForRent	
branchNo		staffNo	branchNo	propertyNo	branchNo
B009		(none for B009)	—	PB01	B009
				PB02	B009
				PB03	B009

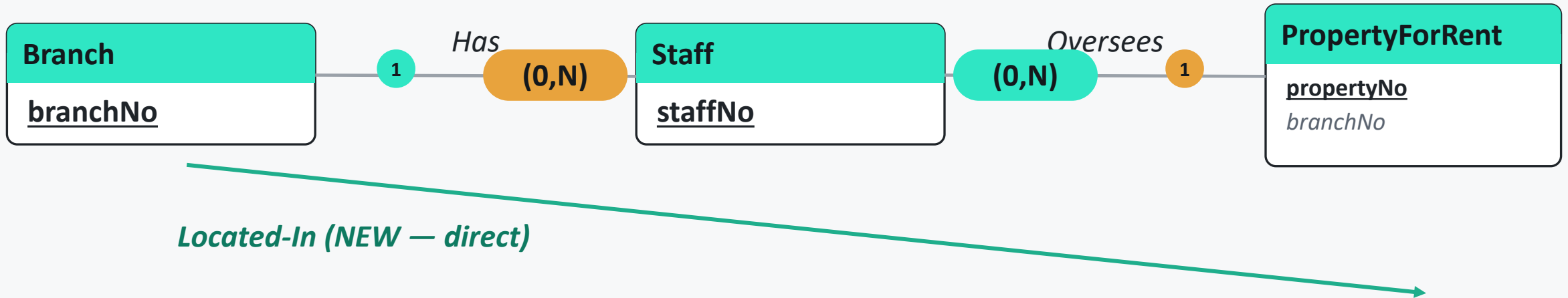
THE WALK-THROUGH

Branch → Staff → PropertyForRent for B009 hits a dead end at step one: zero staff rows, so ZERO properties come back, though three exist. The optional (0,N) step causes it.

THE TELL

Chasm traps make rows silently DISAPPEAR. Fan traps show as wrong pairings.

Chasm Trap — Corrected



WHAT CHANGED AND WHY

What changed, and why: a NEW direct relationship, Branch Located-In PropertyForRent (mandatory on the property side), is added, and PropertyForRent carries branchNo directly. “Which properties belong to B009?” is now answerable straight from PropertyForRent — it never has to pass through the optional Staff hop at all, so a branch with zero staff still correctly returns every property it owns. Both the fan trap and the chasm trap are fixed the identical way: add the missing DIRECT relationship between the two entity types that actually need to be connected, instead of relying on an intermediate path.

KEY TAKEAWAYS

Key Takeaways

- 1 Promote an attribute to its own entity type when it needs its own attributes, its own relationships, or is shared by many occurrences of the entity describing it.
- 2 Synonyms and homonyms aren't just style nits — they silently break joins (synonyms) or silently mix incompatible data (homonyms); the fix is one disambiguated name per concept, applied everywhere.
- 3 A relationship needing its own attributes (Viewing, Registers) deserves first-class treatment, sometimes as a ternary relationship — collapsing a three-way fact into two binary ones can lose the combination.
- 4 Redundant relationships duplicate a fact already derivable via another path — delete the duplicate, never try to keep two copies in sync.
- 5 A fan trap shows up as a WRONG pairing in query results; a chasm trap shows up as rows that silently DISAPPEAR — both are fixed the same way, by adding the missing direct relationship.